Many of the looping constructs that are built into languages such as Java and JavaScript are simply method calls in Ruby, with the methods invoking an associated block zero or more times:

```
["cat", "dog", "horse"].each { |name| print name, " " }
5.times { print "*" }
3.upto(6) { |i| print i }
("a".."e").each { |char| print char }
("a".."e").each { print _1 }
```

produces:

```
cat dog horse *****3456abcdeabcde
```

In the first line, we ask an array to call the block once for each of its elements. Next, the object 5 calls a block five times, printing * each time. Rather than use for loops, the third example shows that in Ruby we can ask the number 3 to call a block, passing in successive values until it reaches 6. Finally, we use Ruby's literal syntax for ranges of values to have the range of characters from *a* to *e* invoke a block using the method each, we show that example twice, once using Ruby's normal block parameter syntax, and the second using Ruby's shortcut for block parameters that we'll see in Blocks, on page 65.

Reading and 'Riting

Ruby comes with a comprehensive library to manage input and output (I/O). However, in most of the examples in this book, we'll stick to a few simple methods. We've already come across methods that write output: puts writes its arguments with a newline after each; p also writes its arguments but will produce more debuggable output. Both can be used to write to any I/O object, but by default they write to the standard output stream.

There are many ways to read input into your program. Probably the most traditional is to use the method gets—short for "get string", which returns the next line from your program's standard input stream:

```
line = gets
print line
```

Because gets returns nil when it reaches the end of input, you can use its return value in a loop condition. Notice that in the following code the condition to the while is an assignment: we store whatever gets returns into the variable line and then test to see whether that returned value was nil or false before continuing:

```
while (line = gets)
  print line
end
```

In Chapter 11, Basic Input and Output, on page 179, we'll talk more about how to read and write from a file or other data source.

Command-Line Arguments

When you run a Ruby program from the command line, you can pass in arguments. These are accessible from your Ruby code in two different ways.

First, the global array ARGV contains each of the arguments passed to the running program. Create a file called cmd_line.rb that contains the following: